Taller para la Casa – User Interface Patterns (Java)

Tema: Patrones de Diseño de Interfaces de Usuario (UI Patterns) y Ética en UI
Autores: Ángela Sofía Puentes Medina y Juan David Parrado Hernández
Modalidad: Trabajo individual o en parejas

Objetivo del taller

Implementar en Java la lógica de estado e interacción de las cuatro categorías de UI Patterns (Navegación, Contenido y Estructura, Entrada de Datos y Feedback y Estado), simulando el flujo de un usuario dentro de una interfaz libre de patrones engañosos (Dark Patterns).

Situación propuesta

Sistema de Configuración y Reservas Universitarias

Una universidad desea renovar su portal de préstamos y reservas de laboratorios y equipos. Actualmente, el sistema antiguo presenta problemas recurrentes de diseño de interfaz:

- Los usuarios se pierden entre menús sin saber en qué sección se encuentran.
- La pantalla muestra decenas de datos técnicos al mismo tiempo, generando sobrecarga visual.
- El formulario rechaza números de teléfono si el estudiante incluye espacios, guiones o el prefijo +57, y exige registrarse antes de permitir armar una reserva.
- El usuario desconoce qué porcentaje de datos le falta completar y no recibe confirmación cuando guarda su solicitud.

El objetivo es construir el controlador de interfaz en Java aplicando un patrón de cada categoría vista en la exposición (la temática puede adaptarse a una tienda virtual, un videojuego o un sistema de comidas, manteniendo las mismas reglas de interacción).

Funcionamiento esperado

1. **Patrón de Navegación (Breadcrumbs + Tabs):** El sistema debe mantener y mostrar en todo momento la ruta jerárquica actual del usuario (por ejemplo: `Inicio > Reservas > Laboratorio de Redes`) a medida que cambia de pestaña o nivel.

2. **Patrón de Contenido y Estructura (Progressive Disclosure / Acordeón):** Al consultar un elemento (por ejemplo, un servidor o equipo), el sistema solo debe mostrar inicialmente su información principal (nombre y estado). Los detalles técnicos secundarios deben permanecer ocultos (`verAvanzado = false`) y solo mostrarse si el usuario ejecuta la acción de desplegarlos.
3. **Patrón de Entrada de Datos (Forgiving Format + Lazy Registration):**
    - **Forgiving Format:** El sistema debe aceptar un teléfono o fecha en cualquier formato (con espacios, paréntesis, guiones o +57) y normalizarlo internamente sin arrojar errores al usuario.
    - **Lazy Registration:** El usuario debe poder armar su reserva sin haber iniciado sesión. Solo cuando intente guardar el pedido final, si no está autenticado, el sistema guardará un borrador temporal y le solicitará crear su cuenta.
4. **Patrón de Feedback y Estado (Completeness Meter + Toast):** El sistema debe calcular dinámicamente el porcentaje de avance del formulario (0%, 33%, 67%, 100%) según los campos completados y emitir una notificación tipo Toast al confirmar el guardado.

Restricción Ética (Cero Dark Patterns)

Está prohibido implementar prácticas de Roach Motel, Confirmshaming, Drip Pricing o Misdirection. Si el usuario decide cancelar su reserva o cerrar sesión, debe poder hacerlo en un solo paso directo y con mensajes neutros.

Requerimientos del ejercicio

El proyecto debe estar organizado por paquetes e incluir, como mínimo, las siguientes clases (los nombres pueden adaptarse a la temática elegida):

Paquete	Clase	Responsabilidad principal
`navigation`	`BreadcrumbNavigator`	Gestiona las pestañas (Tabs) y construye la cadena de ruta (`Inicio > Sección > Subsección`).
`content`	`TarjetaElemento`	Aplica Progressive Disclosure ocultando o revelando los detalles técnicos según el estado booleano.

Paquete	Clase	Responsabilidad principal
`input`	`ValidadorForgiving`	Limpia y normaliza cadenas de entrada (teléfonos, fechas, etc.) usando expresiones regulares sin bloquear al usuario.
`state`	`GestorSesionUI`	Controla el Lazy Registration (borrador temporal), el cálculo del Completeness Meter (%) y el envío de notificaciones Toast.
`main`	`Main`	Ejecuta los casos de prueba demostrando el flujo completo en consola.

Ejemplo esperado de interacción en consola

--- 1. PATRÓN DE NAVEGACIÓN (BREADCRUMBS + TABS) ---

Ruta actual: Inicio > Catálogo > Servidor Principal

--- 2. PATRÓN DE CONTENIDO Y ESTRUCTURA (PROGRESSIVE DISCLOSURE) ---

[Vista Resumen] Servidor Principal · Estado: Activo

[Usuario solicita ver detalles avanzados...]

[Acordeón Desplegado] IP: 192.168.1.10 | Puerto: 8080 | RAM: 4GB/16GB

--- 3. PATRÓN DE ENTRADA DE DATOS (FORGIVING FORMAT) ---

Entrada del usuario: "+57 (300) 123-4567"

Teléfono normalizado sin error: "3001234567"

--- 4. PATRÓN DE FEEDBACK Y ESTADO (COMPLETENESS METER) ---

[Progreso: 67%] Campos listos: 2 de 3 (Falta: Correo de contacto)

Usuario ingresa correo: "estudiante@udistrital.edu.co"

[Progreso: 100%] ¡Formulario completo!

--- 5. LAZY REGISTRATION + TOAST ---

Usuario intenta guardar sin haber iniciado sesión...

[Lazy Registration] Avance guardado en borrador temporal. Por favor cree su cuenta para finalizar.

Usuario completa registro rápido → Sesión activa = true

[Toast de Estado] Reserva guardada exitosamente.

Actividades y Casos de Prueba

- **Parte 1. Diseñar la estructura:** Implementar las clases respetando la organización por paquetes.
- **Parte 2. Programar los 4 UI Patterns:** Asegurar que cada patrón cumpla su función sin usar validaciones rígidas que culpen al usuario.
- **Parte 3. Ejecutar los 5 casos de prueba en Main:** El programa debe incluir en consola al menos estos 5 escenarios verificables:

Caso	Patrón Evaluado	Entrada / Acción	Resultado Esperado en Consola
1	Navegación (Breadcrumb)	Cambiar de pestaña o entrar a una subcategoría	La ruta se actualiza dinámicamente (`Inicio > A > B`).
2	Contenido (Progressive Disclosure)	Consultar tarjeta con `verAvanzado = false` y luego con `true`	Primero oculta los datos técnicos y luego los despliega bajo demanda.
3	Entrada (Forgiving Format)	Ingresar `"+57 (310) 999-8877"`	Retorna `"3109998877"` sin lanzar excepciones ni errores.

Caso	Patrón Evaluado	Entrada / Acción	Resultado Esperado en Consola
4	Feedback (Completeness Meter)	Llenar 1, 2 y 3 campos progresivamente	El medidor sube de 33% a 67% y finalmente a 100%.
5	Entrada + Estado (Lazy Registration + Toast)	Intentar guardar con `usuarioActivo = false` y luego con `true`	Primero conserva el borrador y pide registro; al autenticarse, muestra el Toast de éxito.

Entregable

Entregar un proyecto Java (repositorio) que compile y ejecute correctamente. La entrega debe incluir:

- Código fuente completo organizado por paquetes.
- Captura o texto de la salida en consola con los 5 casos de prueba ejecutados.
- Un párrafo corto explicando cómo su implementación evita caer en Dark Patterns.

Criterios de evaluación (Puntaje total: 50 puntos)

- **10 puntos (20%):** Implementación correcta de los patrones de Navegación (Breadcrumbs/Tabs) y Contenido (Progressive Disclosure).
- **15 puntos (30%):** Implementación correcta de Entrada de Datos (Forgiving Format con limpieza automática y Lazy Registration con borrador temporal).
- **10 puntos (20%):** Implementación correcta de Feedback y Estado (Completeness Meter reactivo y notificaciones Toast).
- **10 puntos (20%):** Los 5 casos de prueba compilan, se ejecutan en consola y el proyecto está bien organizado por paquetes.
- **5 puntos (10%):** Claridad del código, buenas prácticas y reflexión breve sobre ética y prevención de Dark Patterns.